EXPERIMENT-65(HDOC)

C Programming

Program: Multiplication of Two Square Matrices Using Recursion

Step 1: Understand the Problem Statement

- Input: The order n of the square matrices (validated to lie between 1 and 10) and the n × n elements of matrices A and B.
- Output: The resultant n × n matrix C = A × B, displayed row by row.
- Matrix multiplication rule: each element of C is the dot product of a row of A with a column of B, that is `C[i][j] = A[i][0]*B[0][j] + A[i][1]*B[1][j] + ... + A[i][n-1]*B[n-1][j]`.
- No iterative loop is used for the multiplication logic. It is split into two recursive functions: `dotProduct()` recurses over the index k to add up the products for one element, and `multiplyMatrix()` recurses over the column j and then the row i to fill every element of C.
- To keep the whole program loop-free, reading (`readMatrix()`) and displaying (`displayMatrix()`) the matrices are also done recursively, moving across the columns of a row and then down to the next row.

Rule(s) used: C[i][j] = Σ A[i][k] × B[k][j] for k = 0 to n−1. Recursive form: dotProduct(k) = 0 when k = n, else A[row][k] × B[k][col] + dotProduct(k+1). multiplyMatrix(i, j) stops when i = n, moves to the next row when j = n, otherwise sets C[i][j] = dotProduct(i, j, 0) and calls multiplyMatrix(i, j+1).

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result

1	`Order n = 1`
	`A = [[2]]`
	`B = [[5]]`
		[10]	[10]	Pass

2	`Order n = 2`
	`A = [[1, 2], [3, 4]]`
	`B = [[5, 6], [7, 8]]`
		[19, 22]
		[43, 50]
			[19, 22]
			[43, 50]
				Pass

3	`Order n = 3`
	`A = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`
	`B = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]`
		[1, 2, 3]
		[4, 5, 6]
		[7, 8, 9]
			[1, 2, 3]
			[4, 5, 6]
			[7, 8, 9]
				Pass

4	`Order n = 3`
	`A = [[1, 0, 2], [0, 1, 3], [4, 0, 1]]`
	`B = [[2, 1, 0], [1, 3, 1], [0, 2, 2]]`
		[2, 5, 4]
		[1, 9, 7]
		[8, 6, 2]
			[2, 5, 4]
			[1, 9, 7]
			[8, 6, 2]
				Pass

5	`Order n = 0`	Invalid order. Enter a value between 1 and 10.	Invalid order. Enter a value between 1 and 10.	Pass

Step 3: Algorithm

1. Start the program.
2. Read the order n; if n is not between 1 and 10, display an error message and stop.
3. Read the elements of matrix A and then matrix B using the recursive `readMatrix(m, n, i, j, name)`: stop when i = n; when j = n go to the next row; otherwise read `m[i][j]` and call itself with j + 1.
4. Call `multiplyMatrix(A, B, C, n, 0, 0)`.
5. In `multiplyMatrix()`: if i = n, return (base case); if j = n, call itself for the next row (i + 1, 0) and return.
6. Otherwise compute `C[i][j] = dotProduct(A, B, n, i, j, 0)`, where `dotProduct()` returns 0 when k = n, else `A[row][k] * B[k][col]` plus the dot product from k + 1; then call `multiplyMatrix(A, B, C, n, i, j + 1)`.
7. Display matrix C using the recursive `displayMatrix(C, n, 0, 0)`: stop when i = n; at the end of a row print a newline and move to the next row; otherwise print `C[i][j]` and continue with j + 1.
8. Stop the program.

Evaluation against Step 2 test cases: for n = 1, C[0][0] = 2 × 5 = 10. For the 2 × 2 case, C[0][0] = 1·5 + 2·7 = 19, C[0][1] = 1·6 + 2·8 = 22, C[1][0] = 3·5 + 4·7 = 43 and C[1][1] = 3·6 + 4·8 = 50. Multiplying a 3 × 3 matrix by the identity matrix leaves it unchanged, because each dot product picks out exactly one element of A. For the general 3 × 3 case, C[0][0] = 1·2 + 0·1 + 2·0 = 2 and C[2][2] = 4·0 + 0·1 + 1·2 = 2, giving C = [[2, 5, 4], [1, 9, 7], [8, 6, 2]]. An order of 0 is rejected as invalid — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```c
/* HDOC Day-66: Multiply two square matrices using RECURSION.

   C = A x B where A, B and C are n x n matrices:
       C[i][j] = A[i][0]*B[0][j] + A[i][1]*B[1][j] + ... + A[i][n-1]*B[n-1][j]

   No iterative loop (for / while / do-while) is used anywhere in this
   program - reading, multiplying and displaying are all done recursively:
       dotProduct()     recursion over k : sums A[row][k] * B[k][col]
       multiplyMatrix() recursion over j : moves across the columns of a row
                        and over i       : moves down to the next row
*/

#include <stdio.h>

#define MAX 10

/* Function prototypes */
void readMatrix(int m[MAX][MAX], int n, int i, int j, char name);
void displayMatrix(int m[MAX][MAX], int n, int i, int j);
int  dotProduct(int a[MAX][MAX], int b[MAX][MAX], int n, int row, int col, int k);
void multiplyMatrix(int a[MAX][MAX], int b[MAX][MAX], int c[MAX][MAX], int n, int i, int j);

int main() {
    int A[MAX][MAX], B[MAX][MAX], C[MAX][MAX];
    int n;

    printf("Enter order of square matrices (1-%d): ", MAX);
```

```
    if (scanf("%d", &n) != 1 || n < 1 || n > MAX) {
        printf("Invalid order. Enter a value between 1 and %d.\n", MAX);
        return 1;
    }

    printf("Enter elements of matrix A (%d x %d):\n", n, n);
    readMatrix(A, n, 0, 0, 'A');
    printf("Enter elements of matrix B (%d x %d):\n", n, n);
    readMatrix(B, n, 0, 0, 'B');

    multiplyMatrix(A, B, C, n, 0, 0);

    printf("\nResultant matrix C = A x B:\n");
    displayMatrix(C, n, 0, 0);

    return 0;
}

/* Reads element (i, j), then moves to the next column, then to the next row */
void readMatrix(int m[MAX][MAX], int n, int i, int j, char name) {
    if (i == n)                      /* base case: all rows read */
        return;
    if (j == n) {                    /* end of row: go to the next row */
        readMatrix(m, n, i + 1, 0, name);
        return;
    }
    printf("%c[%d][%d] = ", name, i, j);
    scanf("%d", &m[i][j]);
    readMatrix(m, n, i, j + 1, name);
}

/* Displays element (i, j), then the rest of the row, then the next rows */
void displayMatrix(int m[MAX][MAX], int n, int i, int j) {
    if (i == n)                      /* base case: all rows displayed */
        return;
    if (j == n) {                    /* end of row: newline, next row */
        printf("\n");
        displayMatrix(m, n, i + 1, 0);
        return;
    }
    printf("%4d ", m[i][j]);
    displayMatrix(m, n, i, j + 1);
}

/* Returns A[row][k]*B[k][col] + ... + A[row][n-1]*B[n-1][col] */
int dotProduct(int a[MAX][MAX], int b[MAX][MAX], int n, int row, int col, int k) {
    if (k == n)                      /* base case: no more terms to add */
        return 0;
    return a[row][k] * b[k][col] + dotProduct(a, b, n, row, col, k + 1);
}

/* Fills C[i][j], then recursively fills the rest of row i, then the next rows */
void multiplyMatrix(int a[MAX][MAX], int b[MAX][MAX], int c[MAX][MAX], int n, int i, int j) {
    if (i == n)                      /* base case: all rows computed */
        return;
    if (j == n) {                    /* end of row: go to the next row */
        multiplyMatrix(a, b, c, n, i + 1, 0);
        return;
    }
    c[i][j] = dotProduct(a, b, n, i, j, 0);
    multiplyMatrix(a, b, c, n, i, j + 1);
}
```

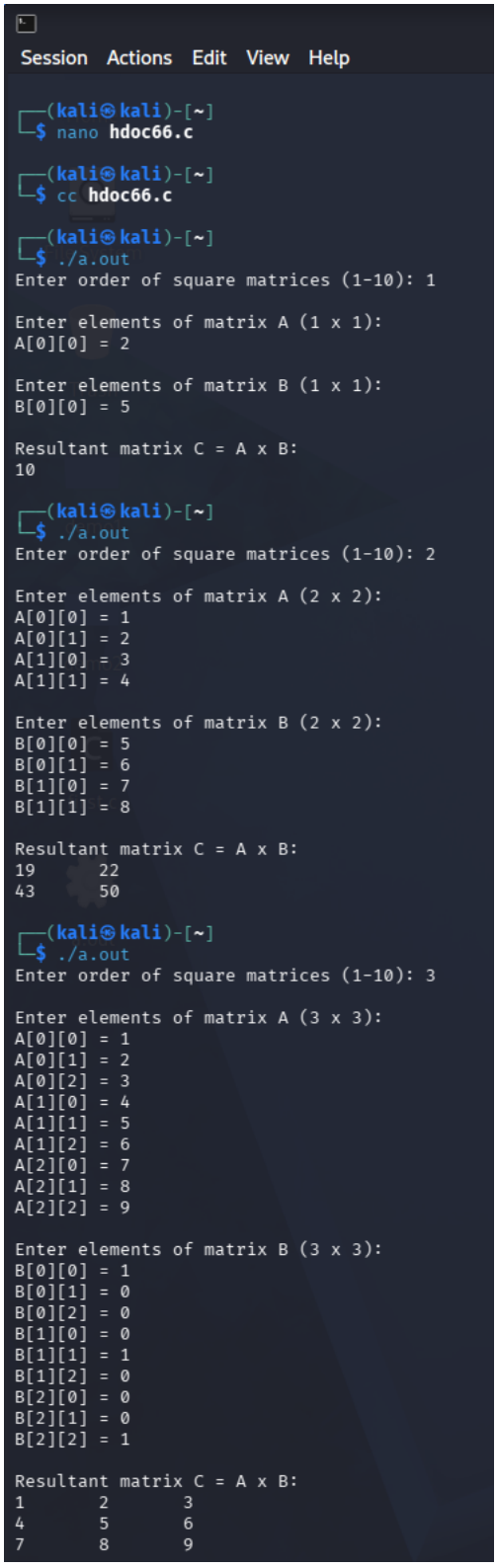
Step 5: Compile, Execute and Evaluate

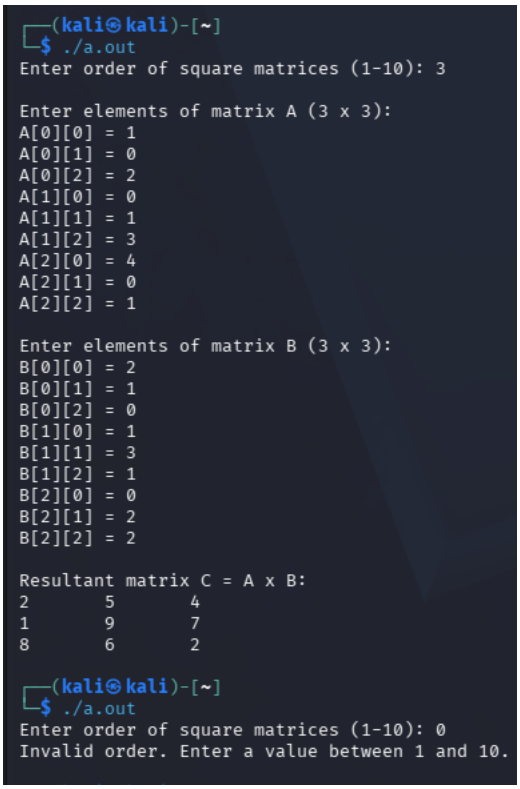
The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

`gcc hdoc66.c`	(compile)
`./a.out`	(execute)

All 5 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot


Session Actions Edit View Help
(kali㉿kali)-[~]
$ nano hdoc66.c
(kali㉿kali)-[~]
$ cc hdoc66.c
(kali㉿kali)-[~]
$ ./a.out
Enter order of square matrices (1-10): 1
Enter elements of matrix A (1 x 1):
A[0][0] = 2
Enter elements of matrix B (1 x 1):
B[0][0] = 5
Resultant matrix C = A x B:
10
(kali㉿kali)-[~]
$ ./a.out
Enter order of square matrices (1-10): 2
Enter elements of matrix A (2 x 2):
A[0][0] = 1
A[0][1] = 2
A[1][0] = 3
A[1][1] = 4
Enter elements of matrix B (2 x 2):
B[0][0] = 5
B[0][1] = 6
B[1][0] = 7
B[1][1] = 8
Resultant matrix C = A x B:
19 22
43 50
(kali㉿kali)-[~]
$ ./a.out
Enter order of square matrices (1-10): 3
Enter elements of matrix A (3 x 3):
A[0][0] = 1
A[0][1] = 2
A[0][2] = 3
A[1][0] = 4
A[1][1] = 5
A[1][2] = 6
A[2][0] = 7
A[2][1] = 8
A[2][2] = 9
Enter elements of matrix B (3 x 3):
B[0][0] = 1
B[0][1] = 0
B[0][2] = 0
B[1][0] = 0
B[1][1] = 1
B[1][2] = 0
B[2][0] = 0
B[2][1] = 0
B[2][2] = 1
Resultant matrix C = A x B:
1 2 3
4 5 6
7 8 9



Conclusion

In this lab session, complete loop-free matrix operations were achieved through mutual and nested recursion in C:

1. Pure Recursive Implementation: Eliminated all standard iterative loops (for, while, do-while) across reading, calculation, and output display functions.

2. Recursive Matrix Multiplication: Implemented dotProduct() to recursively accumulate dot products for individual matrix elements, paired with multiplyMatrix() to step through grid rows and columns (i, j).

3. Execution & Validation: Verified matrix multiplication results across multiple test cases (including identity matrix multiplication and single-element bounds), matching expected output matrix operations cleanly.